

Autonomyx Optimize — Final Multi-Agent System Architecture (with SoD & Human Governance)

1. Overview

Autonomyx Optimize is a multi-agent cost intelligence platform that continuously analyzes, optimizes, and governs technology spend across:

- SaaS applications
- Cloud infrastructure
- LLM / AI usage
- Vendor contracts
- Managed services

It is designed as an **agentic optimization layer** that detects inefficiencies, recommends actions, simulates savings, and executes changes safely through governed workflows.

2. Core Principle

The system answers one continuous question:

Where are we overspending, why, what should we do, and what savings can we unlock safely?

3. Foundational Design Principle: Separation of Duties (SoD)

The system strictly enforces:

Detect → Decide → Do

Layered Responsibility

Layer	Responsibility
Detect	Analyst agents identify issues
Decide	Policy + human approval layer
Do	Execution agent performs actions

Key Rule

No agent that recommends an action can execute it without independent approval.

4. System Architecture Overview

Layer 1: Specialist (Recommender) Agents

These agents are **read-only** and cannot modify systems.

4.1 SaaS Cost Analyst Agent

- Detect unused licenses
- Identify duplicate tools
- Recommend downgrades and consolidation

4.2 Cloud Cost Analyst Agent

- Detect idle resources
- Optimize compute/storage/network usage
- Identify architecture inefficiencies

4.3 LLM Cost Analyst Agent

- Analyze token usage and model selection
- Recommend caching, routing, batching
- Suggest model substitutions

4.4 Vendor Negotiation Agent

- Extract contract terms
- Identify negotiation leverage
- Suggest pricing strategies

4.5 Open-Source Replacement Agent

- Map SaaS → open-source alternatives
- Compare TCO (total cost of ownership)
- Recommend build vs buy decisions

4.6 Skeptic / Challenge Agent (Critical)

- Challenge optimistic savings
 - Validate assumptions
 - Identify hidden costs and risks
-

Layer 2: Governance & Control Layer

4.7 Policy Engine

Determines: - Whether approval is required - Approval thresholds - Risk classification - Auto-approval rules

4.8 Approval Router Agent

- Maps actions to approvers
- Handles multi-step approvals
- Manages escalation and fallback
- Tracks approval states

4.9 Ownership Resolver Agent

- Maps resources → owners
- Maps vendors → responsible teams
- Resolves correct approvers dynamically

4.10 Compliance & Risk Agent

- Validates against security policies
- Checks data sensitivity and residency
- Flags unsafe recommendations

4.11 Audit & Logging Agent

- Maintains immutable logs
- Tracks who approved what
- Ensures compliance traceability

Layer 3: Human Decision Layer

Humans are not replaced — they are **decision authorities**.

Typical roles: - Finance / CFO - Procurement - Engineering / Platform - IT Admin - Security & Compliance - Business Unit Owners

They receive structured approval requests (not raw agent output).

Layer 4: Execution Layer

4.12 Workflow Execution Agent

- Executes approved actions
- Sends vendor communications

- Triggers infrastructure changes
- Creates tickets and workflows

4.13 Rollback / Recovery Agent (Critical)

- Reverts failed or risky actions
 - Restores licenses/resources
 - Ensures system safety
-

Layer 5: Intelligence & Simulation

4.14 Savings Simulation Agent

- Models financial impact
- Calculates ROI and payback
- Produces scenario analysis (best/base/worst)

4.15 Reporting & Narrative Agent

- Generates executive summaries
 - Produces dashboards and reports
 - Tailors insights for different stakeholders
-

5. Approval Architecture (Human-in-the-Loop)

Flow

1. Agent generates recommendation
 2. Policy Engine evaluates requirement
 3. Approval Router identifies approver(s)
 4. Human reviews structured recommendation
 5. Decision recorded
 6. Execution triggered (if approved)
-

Approval State Machine

- Drafted
- Awaiting Approval
- Under Review
- Approved
- Rejected
- Expired
- Escalated
- Executed

- Rolled Back
-

Approval Rules Examples

- Low-risk (< \$1K impact): auto-approved
 - Medium-risk: team manager approval
 - High-risk: manager + finance + procurement
 - Production infra change: engineering + security
 - Contract change: procurement + finance
-

6. Data & Knowledge Layer

6.1 Enterprise Knowledge Graph

Entities: - Vendors - Tools - Teams - Users - Contracts - Cloud resources - AI workflows - Costs - Recommendations

6.2 Operational Memory

- Active tasks
- Pending approvals
- Recent analyses

6.3 Policy Memory

- Approval thresholds
- Compliance rules
- Preferred vendors

6.4 Outcome Memory

- Accepted/rejected actions
 - Learning from decisions
-

7. Security & SoD Enforcement

Access Control

Agent Type	Access
Analyst Agents	Read-only
Governance Agents	Policy + metadata

Agent Type	Access
Execution Agent	Scoped write access

Additional Controls

- Role-based access control (RBAC)
- Immutable audit logs
- Zero-trust execution model
- Credential separation per agent

8. End-to-End Workflow Example

Scenario: Reduce 15% Cost

1. User triggers optimization goal
2. Data ingestion refreshes all sources
3. Analyst agents detect inefficiencies
4. Skeptic agent validates findings
5. Simulation agent ranks opportunities
6. Policy engine evaluates approval needs
7. Approval router sends requests
8. Humans approve/reject
9. Execution agent implements approved actions
10. Rollback agent safeguards failures
11. Reporting agent generates summary

9. MVP Agent Set

Start with: 1. Orchestrator Agent 2. Data Ingestion Agent 3. SaaS Cost Analyst 4. Cloud + LLM Analyst 5. Reporting Agent

Then expand to: - Policy Engine - Approval Router - Execution Agent - Simulation Agent - Skeptic Agent

10. Product Modules

10.1 Optimize Control Tower

Unified cost visibility + recommendations

10.2 Renewal Intelligence

Vendor negotiation workspace

10.3 AI Spend Optimizer

LLM and AI cost analysis

10.4 Open Alternative Advisor

Open-source replacement engine

10.5 Action Center

Approvals, execution, tracking

11. Recommendation Output Model

Each recommendation includes:

- Problem detected
 - Evidence
 - Current cost
 - Recommended action
 - Expected savings
 - Implementation cost
 - Payback period
 - Confidence score
 - Risk score
 - Required approvals
 - Affected owners
-

12. North Star Metrics

- Identified savings
 - Realized savings
 - Approval rate
 - Time to execution
 - Payback time
 - Spend under management
 - Reduction in waste (SaaS/cloud/LLM)
-

13. Positioning

Autonomyx Optimize is not a dashboard.

It is a **governed multi-agent system** that: - detects inefficiencies - recommends actions - enforces approval policies - executes safely - continuously learns

14. Final Definition

Autonomyx Optimize is a governed multi-agent cost intelligence system that reduces SaaS, cloud, and AI spend through analysis, policy-driven approvals, and safe execution.

15. Next Steps — Detailed Implementation Plan

Phase 1: Define the Common Spend Ontology

Build a unified ontology so all agents reason over the same entities and relationships.

Core Entities

- Organization
- Business Unit
- Team
- User
- Vendor
- Product
- Contract
- Subscription
- License
- Invoice
- Cloud Resource
- Environment
- AI Workflow
- Model
- Cost Event
- Recommendation
- Approval
- Execution Job
- Savings Outcome

Core Relationships

- Team owns Product
- Vendor supplies Product
- Contract governs Subscription
- Subscription contains Licenses
- Team owns Cloud Resource
- AI Workflow uses Model
- Recommendation affects Resource / Contract / Subscription

- Approval authorizes Execution Job
- Execution Job produces Savings Outcome

Design Rule

Every agent must read and write against this shared ontology, not vendor-specific terms.

Phase 2: Build Ingestion Pipelines

Required Connectors

- SaaS admin APIs
- Cloud billing and usage APIs
- SSO / IdP logs
- ERP / finance systems
- Procurement systems
- Contract repositories
- LLM gateway / observability tools
- Manual CSV / PDF import for bootstrap

Pipeline Stages

1. Fetch raw source data
2. Normalize to common schema
3. Resolve identities and ownership
4. Deduplicate vendors / products / contracts
5. Compute freshness score
6. Store structured records + raw artifacts

Output

A normalized spend graph plus event streams for usage, billing, and approvals.

Phase 3: Implement Agent Orchestration

Use a central orchestrator that turns a goal into a governed workflow.

Orchestrator Responsibilities

- Accept goals, alerts, or schedules
- Decompose work into subtasks
- Call specialist agents in parallel where safe
- Merge outputs
- Trigger policy checks
- Pause for approvals
- Resume after decisions

- Trigger execution and post-execution verification

Recommended State Flow

- Intake
 - Data Refresh
 - Analysis
 - Challenge Review
 - Simulation
 - Policy Check
 - Approval Routing
 - Execution
 - Verification
 - Reporting
 - Outcome Learning
-

Phase 4: Build the Policy + Approval System

Core Components

- Policy Engine
- Approval Router
- Ownership Resolver
- Audit Logger

Policy Dimensions

- Financial threshold
- Operational risk
- Environment type
- Data sensitivity
- Vendor / contract impact
- Security impact
- Business criticality

Example Policy Logic

- If annual savings < \$1K and risk is low → auto-approve
 - If SaaS cancellation impacts multiple teams → manager approval
 - If production cloud change → engineering owner + security
 - If contract term changes → procurement + finance
-

Phase 5: Launch Core Analyst Agents

Minimum Production Agents

- SaaS Cost Analyst
- Cloud Cost Analyst
- LLM Cost Analyst
- Skeptic / Challenge Agent
- Savings Simulation Agent
- Reporting Agent

Early Deliverables

- License waste detection
 - Idle cloud resource detection
 - LLM token and model waste analysis
 - Ranked recommendation list
 - Executive summary output
-

Phase 6: Add Safe Execution + Rollback

Execution Capabilities

- Draft actions only at first
- Then enable scoped automation for low-risk actions
- Expand to write actions after audit and policy maturity

Rollback Requirements

- Every execution plan must include rollback steps
 - Rollback must be tested for recurring workflows
 - System should support partial rollback and manual override
-

Phase 7: Expand Product Depth

Add-on Agents

- Vendor Negotiation Agent
- Open-Source Replacement Agent
- Compliance & Risk Agent
- Forecasting Agent
- Benchmarking Agent

Expansion Modules

- Renewal intelligence

- Build-vs-buy analysis
 - Continuous optimization digests
 - Budget forecasting and anomaly alerts
-

16. Target Technical Architecture

16.1 Core Services

- **API Gateway** — entry point for UI, workflows, and external integrations
- **Orchestrator Service** — runs multi-agent workflows
- **Policy Service** — evaluates approval and risk rules
- **Approval Service** — manages approval requests and state transitions
- **Execution Service** — triggers approved actions
- **Rollback Service** — reverts failed changes
- **Reporting Service** — generates summaries and dashboards

16.2 Data Stores

- **Postgres** — operational data, recommendations, approvals, jobs
- **Object Store** — contracts, invoices, exports, logs, snapshots
- **Graph Store / Graph Layer** — ownership and relationship reasoning
- **Vector Store** — contract, policy, and document retrieval
- **Event Bus** — asynchronous workflow events

16.3 Recommended Internal Components

- Connector workers
 - Identity / ownership resolution service
 - Cost normalization engine
 - Recommendation ranking engine
 - Audit log pipeline
 - Notification service
-

17. Reference API Design

17.1 Goal Intake API

```
POST /optimize/goals
{
  "goal_type": "cost_reduction",
  "target_scope": ["saas", "cloud", "llm"],
  "target_savings_percent": 15,
  "time_horizon_days": 90,
  "constraints": {
```

```
    "exclude_production_shutdowns": true,  
    "require_human_approval": true  
  }  
}
```

17.2 Recommendation API

```
GET /recommendations/{recommendation_id}  
{  
  "id": "rec_123",  
  "category": "saas",  
  "problem": "214 inactive paid seats for 60+ days",  
  "evidence": [  
    "No login activity detected in 68 days",  
    "Seat assigned to inactive users"  
  ],  
  "current_cost_annual": 120000,  
  "recommended_action": "remove_180_seats_and_downgrade_34",  
  "expected_savings_annual": 86400,  
  "implementation_cost": 2000,  
  "payback_days": 9,  
  "risk_score": "low",  
  "confidence_score": "high",  
  "required_approvals": ["revops_manager"],  
  "status": "awaiting_approval"  
}
```

17.3 Approval API

```
POST /approvals/{approval_id}/decision  
{  
  "decision": "approved",  
  "comment": "Proceed after notifying team leads",  
  "approved_by": "user_456"  
}
```

17.4 Execution API

```
POST /executions  
{  
  "recommendation_id": "rec_123",  
  "approved_approval_ids": ["apr_789"],  
}
```

```
    "execution_mode": "controlled_live_run"
}
```

17.5 Rollback API

```
POST /executions/{execution_id}/rollback
{
  "reason": "Unexpected access loss reported by team"
}
```

18. Agent Message Schemas

18.1 Shared Agent Envelope

```
{
  "message_id": "msg_001",
  "workflow_id": "wf_001",
  "goal_id": "goal_001",
  "agent": "saas_cost_analyst",
  "message_type": "recommendation",
  "timestamp": "2026-04-09T10:00:00Z",
  "payload": {}
}
```

18.2 Recommendation Payload

```
{
  "recommendation_id": "rec_123",
  "category": "saas",
  "scope": {
    "vendor": "ExampleCRM",
    "team": "Revenue Operations"
  },
  "issue_type": "inactive_licenses",
  "summary": "214 inactive paid seats detected",
  "evidence": [
    {
      "type": "usage_signal",
      "detail": "No login activity for 60+ days"
    }
  ],
  "recommended_actions": [
```

```

    {
      "action_type": "remove_license",
      "count": 180
    },
    {
      "action_type": "downgrade_plan",
      "count": 34
    }
  ],
  "estimated_savings": {
    "annual": 86400,
    "currency": "USD"
  },
  "implementation_cost": 2000,
  "risk": "low",
  "confidence": "high",
  "needs_policy_review": true
}

```

18.3 Approval Request Payload

```

{
  "approval_id": "apr_789",
  "recommendation_id": "rec_123",
  "required_roles": ["revops_manager"],
  "approval_reason": "License reduction above team threshold",
  "decision_deadline": "2026-04-15T17:00:00Z",
  "execution_blocked_until_approved": true
}

```

18.4 Execution Job Payload

```

{
  "execution_id": "exe_321",
  "recommendation_id": "rec_123",
  "execution_plan": [
    "Notify impacted owners",
    "Remove 180 inactive seats",
    "Downgrade 34 seats",
    "Verify access integrity"
  ],
  "rollback_plan": [
    "Restore removed seats from snapshot",
    "Re-assign previous license tier",
    "Notify owners of rollback"
  ]
}

```

```

    ],
    "execution_risk": "low"
  }

```

19. UI / Product Workflows

19.1 Control Tower

Shows: - spend by category - top recommendations - risk-weighted savings - pending approvals - realized savings

19.2 Approval Inbox

Shows: - what changed - why it matters - expected savings - risks - rollback plan - approve / reject / defer actions

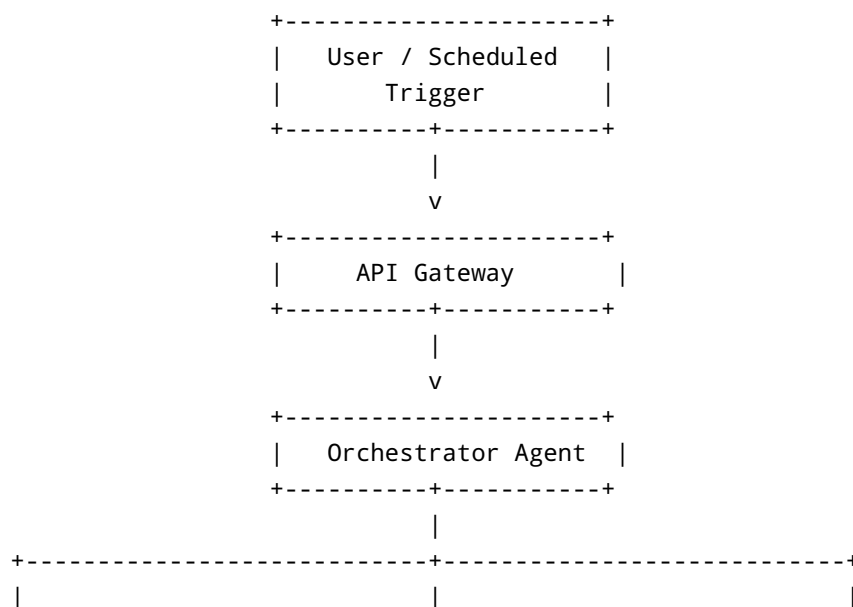
19.3 Recommendation Workspace

Shows: - evidence - challenger comments - simulation scenarios - owner mapping - downstream impact

19.4 Execution Center

Shows: - approved jobs - live execution state - verification checks - rollback controls

20. Visual Architecture Diagram (Text Blueprint)





21. Delivery Roadmap

Sprint 1-2

- ontology v1
- manual CSV ingestion
- SaaS license waste analysis
- recommendation object model

Sprint 3-4

- cloud billing ingestion
- LLM cost ingestion
- orchestrator workflow engine
- reporting summary v1

Sprint 5-6

- policy engine v1
- approval router v1
- approval inbox UI
- audit logging

Sprint 7-8

- controlled execution for low-risk SaaS actions
- rollback workflows
- simulation engine v1
- skeptic agent v1

Sprint 9-12

- vendor negotiation module
- open-source replacement module
- forecasting and anomaly detection
- enterprise RBAC and tenant hardening

22. Final Thought

The real innovation is not just discovering savings.

It is building a system that can **reason, challenge itself, request approval, execute safely, and prove what happened.**

That is what makes **Autonomyx Optimize** enterprise-grade and defensible.

23. Product Requirements Document (PRD)

23.1 Product Overview

Autonomyx Optimize is a governed multi-agent system that reduces SaaS, cloud, and AI spend through continuous analysis, policy-driven approvals, and safe execution.

23.2 Target Users

Primary Users

- CFO / Finance teams
- Procurement teams
- Engineering / Platform teams
- IT administrators

Secondary Users

- Security & Compliance teams
 - Business unit leaders
-

23.3 User Problems

- Lack of visibility into SaaS, cloud, and AI costs
 - High unused or underutilized resources
 - Poor vendor negotiation outcomes
 - No structured way to optimize LLM usage
 - Risk of unsafe automation in cost reduction
-

23.4 Core Features

1. Cost Visibility Dashboard

- Unified view of SaaS, cloud, and LLM spend
- Drill-down by team, vendor, environment

2. Recommendation Engine

- Detect inefficiencies
- Generate actionable recommendations
- Provide evidence and impact

3. Approval Workflow System

- Policy-based approvals
- Multi-step approval routing
- Approval inbox UI

4. Execution Engine

- Execute approved actions
- Track execution status
- Provide rollback capability

5. Savings Simulation

- Estimate savings and ROI
- Scenario modeling

6. Reporting

- Executive summaries
 - Weekly optimization reports
-

23.5 User Stories

CFO

- As a CFO, I want to see total waste across the stack so that I can reduce budget leakage.

Procurement

- As procurement, I want negotiation insights so that I can reduce vendor contracts.

Engineer

- As an engineer, I want to identify inefficient workloads so that I can optimize infrastructure.

IT Admin

- As IT admin, I want to detect unused SaaS licenses so that I can reduce subscriptions.
-

23.6 Non-Functional Requirements

Performance

- Recommendations generated within minutes for medium datasets

Scalability

- Handle enterprises with 1000+ SaaS tools and multi-cloud setups

Security

- RBAC enforcement
- Data encryption at rest and in transit

Reliability

- 99.9% uptime for core services

Auditability

- Full trace of recommendations, approvals, executions
-

24. System Design Specification

24.1 High-Level Architecture

Components: - API Gateway - Orchestrator - Agent Services - Policy Engine - Approval Service - Execution Service - Data Layer

24.2 Data Model (Simplified)

Tables: - organizations - teams - users - vendors - products - contracts - subscriptions - resources - cost_events - recommendations - approvals - executions

24.3 RBAC Model

Roles: - Admin - Finance - Procurement - Engineer - Viewer

Permissions: - Read data - Approve actions - Execute actions - Configure policies

24.4 Event-Driven Architecture

Event Types: - recommendation_created - approval_requested - approval_granted - execution_started - execution_completed - rollback_triggered

24.5 Failure Handling

- Retry failed ingestion jobs
 - Fallback approvers for approval timeouts
 - Automatic rollback on execution failure
-

24.6 Security Design

- Zero-trust access between services
 - Token-based authentication
 - Audit logs for all actions
-

24.7 Deployment Model

- Microservices architecture
 - Containerized services (Kubernetes)
 - Cloud-agnostic deployment
-

24.8 Observability

- Logs: all agent actions
 - Metrics: savings, execution success
 - Traces: workflow execution paths
-

24.9 Future Enhancements

- Autonomous negotiation agents
 - Predictive cost forecasting
 - Benchmarking against industry peers
-

Final Note

This PRD + System Design provides a complete blueprint to build Autonomyx Optimize as a scalable, secure, and enterprise-grade multi-agent platform.

25. Database Schema (Actual SQL Tables)

Below is a practical Postgres-oriented schema for the MVP and near-term production system.

25.1 Organizations

```
CREATE TABLE organizations (  
  id UUID PRIMARY KEY,  
  name TEXT NOT NULL,  
  slug TEXT UNIQUE NOT NULL,  
  status TEXT NOT NULL DEFAULT 'active',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

25.2 Business Units

```
CREATE TABLE business_units (  
  id UUID PRIMARY KEY,  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  code TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (organization_id, name)  
);
```

25.3 Teams

```
CREATE TABLE teams (  
  id UUID PRIMARY KEY,  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  
  business_unit_id UUID REFERENCES business_units(id) ON DELETE SET NULL,  
  name TEXT NOT NULL,  
  manager_user_id UUID,  
  cost_center TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (organization_id, name)  
);
```

25.4 Users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  
  email TEXT NOT NULL,  
  full_name TEXT,  
  status TEXT NOT NULL DEFAULT 'active',  
  idp_external_id TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (organization_id, email)  
);
```

25.5 Roles and User Roles

```
CREATE TABLE roles (  
  id UUID PRIMARY KEY,  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  
  role_key TEXT NOT NULL,  
  role_name TEXT NOT NULL,  
  UNIQUE (organization_id, role_key)  
);  
  
CREATE TABLE user_roles (  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,  
  assigned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  PRIMARY KEY (user_id, role_id)  
);
```

25.6 Vendors and Products

```
CREATE TABLE vendors (  
  id UUID PRIMARY KEY,  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  normalized_name TEXT NOT NULL,  
  category TEXT,  
  website TEXT,  
  status TEXT NOT NULL DEFAULT 'active',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

    UNIQUE (organization_id, normalized_name)
);

CREATE TABLE products (
  id UUID PRIMARY KEY,
  vendor_id UUID NOT NULL REFERENCES vendors(id) ON DELETE CASCADE,
  product_name TEXT NOT NULL,
  product_type TEXT NOT NULL,
  sku TEXT,
  deployment_model TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (vendor_id, product_name, COALESCE(sku, ''))
);

```

25.7 Contracts and Subscriptions

```

CREATE TABLE contracts (
  id UUID PRIMARY KEY,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  vendor_id UUID NOT NULL REFERENCES vendors(id) ON DELETE CASCADE,
  contract_number TEXT,
  start_date DATE,
  end_date DATE,
  auto_renew BOOLEAN NOT NULL DEFAULT FALSE,
  renewal_notice_days INTEGER,
  annual_commit_amount NUMERIC(18,2),
  currency_code CHAR(3) NOT NULL DEFAULT 'USD',
  owner_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  source_document_uri TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE subscriptions (
  id UUID PRIMARY KEY,
  contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
  billing_frequency TEXT,
  plan_name TEXT,
  seat_count INTEGER,
  unit_price NUMERIC(18,4),
  status TEXT NOT NULL DEFAULT 'active',
  start_date DATE,

```

```

    end_date DATE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

25.8 Licenses

```

CREATE TABLE licenses (
    id UUID PRIMARY KEY,
    subscription_id UUID NOT NULL REFERENCES subscriptions(id) ON DELETE CASCADE,
    assigned_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    assigned_team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
    license_type TEXT,
    status TEXT NOT NULL DEFAULT 'active',
    last_activity_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

25.9 Cloud Resources

```

CREATE TABLE cloud_resources (
    id UUID PRIMARY KEY,
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    provider TEXT NOT NULL,
    account_id TEXT,
    region TEXT,
    resource_type TEXT NOT NULL,
    resource_identifier TEXT NOT NULL,
    environment TEXT,
    owner_team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
    status TEXT NOT NULL DEFAULT 'active',
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (organization_id, provider, resource_identifier)
);

```

25.10 AI Workflows and Models

```

CREATE TABLE ai_models (
    id UUID PRIMARY KEY,
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,

```

```

provider TEXT NOT NULL,
model_name TEXT NOT NULL,
model_family TEXT,
hosting_type TEXT,
price_input_per_million NUMERIC(18,6),
price_output_per_million NUMERIC(18,6),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (organization_id, provider, model_name)
);

CREATE TABLE ai_workflows (
  id UUID PRIMARY KEY,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  workflow_name TEXT NOT NULL,
  owner_team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
  business_criticality TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (organization_id, workflow_name)
);

```

25.11 Cost Events

```

CREATE TABLE cost_events (
  id UUID PRIMARY KEY,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  source_type TEXT NOT NULL,
  source_ref_id TEXT,
  vendor_id UUID REFERENCES vendors(id) ON DELETE SET NULL,
  subscription_id UUID REFERENCES subscriptions(id) ON DELETE SET NULL,
  cloud_resource_id UUID REFERENCES cloud_resources(id) ON DELETE SET NULL,
  ai_workflow_id UUID REFERENCES ai_workflows(id) ON DELETE SET NULL,
  ai_model_id UUID REFERENCES ai_models(id) ON DELETE SET NULL,
  team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
  event_date DATE NOT NULL,
  cost_amount NUMERIC(18,6) NOT NULL,
  currency_code CHAR(3) NOT NULL DEFAULT 'USD',
  usage_quantity NUMERIC(18,6),
  usage_unit TEXT,
  raw_payload JSONB NOT NULL DEFAULT '{} '::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_cost_events_org_event_date ON cost_events (organization_id,
event_date);

```

```
CREATE INDEX idx_cost_events_vendor ON cost_events (vendor_id);
CREATE INDEX idx_cost_events_team ON cost_events (team_id);
```

25.12 Recommendations

```
CREATE TABLE recommendations (
  id UUID PRIMARY KEY,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  category TEXT NOT NULL,
  title TEXT NOT NULL,
  summary TEXT NOT NULL,
  issue_type TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'drafted',
  risk_score TEXT NOT NULL,
  confidence_score TEXT NOT NULL,
  estimated_savings_annual NUMERIC(18,2),
  implementation_cost NUMERIC(18,2),
  payback_days INTEGER,
  evidence JSONB NOT NULL DEFAULT '[]'::jsonb,
  scope JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_by_agent TEXT NOT NULL,
  challenged_by_agent TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_recommendations_org_status ON recommendations
(organization_id, status);
CREATE INDEX idx_recommendations_category ON recommendations (organization_id,
category);
```

25.13 Approvals

```
CREATE TABLE approvals (
  id UUID PRIMARY KEY,
  recommendation_id UUID NOT NULL REFERENCES recommendations(id) ON DELETE
CASCADE,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  approval_stage INTEGER NOT NULL DEFAULT 1,
  required_role_key TEXT,
  approver_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  decision TEXT NOT NULL DEFAULT 'pending',
  decision_comment TEXT,
  decision_at TIMESTAMPTZ,
```

```

decision_deadline TIMESTAMPTZ,
escalated_to_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_approvals_recommendation ON approvals (recommendation_id);
CREATE INDEX idx_approvals_approver_status ON approvals (approver_user_id,
decision);

```

25.14 Execution Jobs

```

CREATE TABLE execution_jobs (
  id UUID PRIMARY KEY,
  recommendation_id UUID NOT NULL REFERENCES recommendations(id) ON DELETE
CASCADE,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  execution_mode TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'queued',
  execution_plan JSONB NOT NULL DEFAULT '[]'::jsonb,
  rollback_plan JSONB NOT NULL DEFAULT '[]'::jsonb,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  failure_reason TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

25.15 Savings Outcomes

```

CREATE TABLE savings_outcomes (
  id UUID PRIMARY KEY,
  recommendation_id UUID NOT NULL REFERENCES recommendations(id) ON DELETE
CASCADE,
  execution_job_id UUID REFERENCES execution_jobs(id) ON DELETE SET NULL,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  projected_savings_annual NUMERIC(18,2),
  realized_savings_annual NUMERIC(18,2),
  measurement_period_start DATE,
  measurement_period_end DATE,
  outcome_status TEXT NOT NULL DEFAULT 'tracking',
  notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

25.16 Audit Logs

```
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY,
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  actor_type TEXT NOT NULL,
  actor_id TEXT,
  action_type TEXT NOT NULL,
  target_type TEXT NOT NULL,
  target_id TEXT NOT NULL,
  metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_logs_org_created_at ON audit_logs (organization_id,
created_at DESC);
```

26. Agent-Level Prompts + Tool Definitions

These are implementation-oriented starting prompts and tool contracts for each core agent.

26.1 Orchestrator Agent

System Prompt

You are the Orchestrator Agent for Autonomyx Optimize. Your job is to convert a user goal or scheduled trigger into a governed workflow. Decompose work, call specialist agents, merge outputs, send results to policy evaluation, pause for approvals, and route approved items to execution. Never fabricate evidence. Never bypass approval requirements.

Inputs

- goal
- scope
- constraints
- freshness requirements

Outputs

- workflow plan

- task assignments
- merged findings
- execution requests

Tools

- `start_data_refresh(scope)`
 - `invoke_agent(agent_name, payload)`
 - `evaluate_policy(recommendation_id)`
 - `create_approval_request(recommendation_id)`
 - `create_execution_job(recommendation_id)`
 - `write_audit_log(action, target)`
-

26.2 SaaS Cost Analyst Agent

System Prompt

You are the SaaS Cost Analyst Agent. Identify waste, duplication, under-utilization, and downgrade opportunities across SaaS products. Use only available evidence. Provide quantified savings estimates, evidence, risk, and a structured recommendation. You are read-only and cannot trigger execution.

Tools

- `query_subscriptions(filters)`
- `query_licenses(filters)`
- `query_cost_events(filters)`
- `query_usage_activity(filters)`
- `create_recommendation(payload)`

Output Shape

- title
 - issue_type
 - impacted_vendor
 - impacted_team
 - evidence
 - estimated_savings
 - risk
 - confidence
-

26.3 Cloud Cost Analyst Agent

System Prompt

You are the Cloud Cost Analyst Agent. Detect idle resources, sizing inefficiencies, environment scheduling opportunities, wasteful data transfer patterns, and architecture-driven cost issues. Provide conservative savings estimates and operational risk notes. Never recommend production-disruptive action without explicitly flagging risk.

Tools

- `query_cloud_resources(filters)`
 - `query_cost_events(filters)`
 - `query_utilization_metrics(filters)`
 - `create_recommendation(payload)`
-

26.4 LLM Cost Analyst Agent

System Prompt

You are the LLM Cost Analyst Agent. Analyze model usage, prompt inefficiency, repeated calls, routing gaps, and caching opportunities. Recommend ways to lower AI spend while preserving acceptable quality. Distinguish between low-risk optimizations and quality-sensitive changes.

Tools

- `query_ai_workflows(filters)`
 - `query_model_usage(filters)`
 - `query_cost_events(filters)`
 - `create_recommendation(payload)`
-

26.5 Vendor Negotiation Agent

System Prompt

You are the Vendor Negotiation Agent. Analyze vendor contracts, renewal windows, actual usage, and pricing commitments. Generate negotiation strategies, leverage points, and fallback positions. Do not send communications directly unless routed through an approved execution workflow.

Tools

- `query_contracts(filters)`
- `extract_contract_terms(contract_id)`
- `query_subscriptions(filters)`

- `query_cost_events(filters)`
 - `create_recommendation(payload)`
 - `draft_vendor_memo(payload)`
-

26.6 Open-Source Replacement Agent

System Prompt

You are the Open-Source Replacement Agent. Evaluate where proprietary tools or managed services can be replaced by open-source or self-hosted alternatives. Compare ongoing SaaS cost, implementation cost, operational burden, compliance needs, and migration risk. Avoid naive recommendations that ignore operating overhead.

Tools

- `query_products(filters)`
 - `query_subscriptions(filters)`
 - `lookup_open_source_candidates(category)`
 - `estimate_tco(payload)`
 - `create_recommendation(payload)`
-

26.7 Skeptic / Challenge Agent

System Prompt

You are the Skeptic Agent. Challenge recommendations created by other agents. Look for overstated savings, hidden migration cost, weak evidence, missing downstream risk, or unrealistic execution assumptions. You do not create final actions; you improve trustworthiness.

Tools

- `query_recommendation(recommendation_id)`
 - `query_related_evidence(recommendation_id)`
 - `append_challenge_notes(recommendation_id, payload)`
-

26.8 Savings Simulation Agent

System Prompt

You are the Savings Simulation Agent. Translate recommendations into quantified scenarios. Compute projected savings, implementation cost, payback period, and best/base/worst cases. Be conservative when confidence is low.

Tools

- `query_recommendation(recommendation_id)`
 - `query_historical_costs(scope)`
 - `update_recommendation_financials(recommendation_id, payload)`
-

26.9 Policy Engine Agent

System Prompt

You are the Policy Engine Agent. Determine whether a recommendation requires approval, who must approve it, whether it is auto-approvable, and which controls apply. You must enforce policy consistently and never bypass constraints.

Tools

- `query_recommendation(recommendation_id)`
 - `query_policy_rules(filters)`
 - `query_ownership(scope)`
 - `create_policy_evaluation(payload)`
-

26.10 Approval Router Agent

System Prompt

You are the Approval Router Agent. Resolve the correct approvers for a recommendation based on policy, ownership, role mapping, business criticality, and fallback rules. Track state transitions and escalations.

Tools

- `query_policy_evaluation(recommendation_id)`
 - `query_users_by_role(role_key, scope)`
 - `create_approval_record(payload)`
 - `escalate_approval(approval_id, user_id)`
-

26.11 Workflow Execution Agent

System Prompt

You are the Workflow Execution Agent. Execute only actions that have valid approvals or are explicitly auto-approved by policy. Follow the approved execution plan, verify outcomes, write audit logs, and stop immediately on unexpected risk signals.

Tools

- `query_execution_job(execution_job_id)`
 - `execute_saas_action(payload)`
 - `execute_cloud_action(payload)`
 - `execute_notification(payload)`
 - `mark_execution_status(execution_job_id, status)`
 - `write_audit_log(action, target)`
 - `trigger_rollback(execution_job_id)`
-

26.12 Rollback Agent

System Prompt

You are the Rollback Agent. Restore systems to the last safe known state after a failed or unsafe execution. Prefer reversible, well-logged remediation. Never hide failed actions.

Tools

- `query_execution_job(execution_job_id)`
 - `run_rollback_step(payload)`
 - `mark_execution_status(execution_job_id, status)`
 - `write_audit_log(action, target)`
-

26.13 Reporting Agent

System Prompt

You are the Reporting Agent. Convert recommendations, approvals, execution outcomes, and savings into executive-friendly and operator-friendly summaries. Be concise, factual, and role-aware.

Tools

- `query_recommendations(filters)`
 - `query_approvals(filters)`
 - `query_execution_jobs(filters)`
 - `query_savings_outcomes(filters)`
 - `publish_report(payload)`
-

27. Policy DSL (How Rules Are Written)

The policy DSL should be simple, auditable, and machine-evaluable.

27.1 Design Goals

- human-readable
- easy to diff in git
- supports conditions + actions
- supports approval chains
- supports auto-approval thresholds

27.2 Example DSL Structure

```
policy_id: saas_license_removal_low_risk
version: 1
enabled: true
applies_to:
  category: saas
  issue_type: inactive_licenses
conditions:
  risk_score: low
  estimated_savings_annual_lt: 1000
  affects_production: false
decision:
  mode: auto_approve
  execution_mode: controlled_live_run
```

27.3 Approval Rule Example

```
policy_id: cloud_production_change
version: 1
enabled: true
applies_to:
  category: cloud
conditions:
  affects_production: true
decision:
  mode: require_approval
  approval_chain:
    - role: engineering_owner
    - role: security_reviewer
  execution_mode: manual_after_approval
```

27.4 Contract Rule Example

```
policy_id: vendor_contract_change
version: 1
enabled: true
applies_to:
  category: vendor
  issue_type: contract_renegotiation
conditions:
  annual_commit_amount_gte: 50000
decision:
  mode: require_approval
  approval_chain:
    - role: procurement_owner
    - role: finance_owner
  execution_mode: draft_only
```

27.5 Recommended DSL Fields

```
policy_id:
version:
enabled:
description:
applies_to:
conditions:
decision:
exceptions:
metadata:
```

27.6 Semantics

`applies_to`

Defines matching scope: - category - issue_type - product_type - environment

`conditions`

Supported condition examples: - `risk_score` - `confidence_score` -
`estimated_savings_annual_lt` - `estimated_savings_annual_gte` - `implementation_cost_lt` -
`affects_production` - `contains_sensitive_data` - `business_criticality_in`

`decision.mode`

Allowed modes: - `auto_approve` - `require_approval` - `deny` - `draft_only`

approval_chain

Ordered sequence of required roles.

27.7 Composite Rule Example

```
policy_id: open_source_replacement_sensitive_system
version: 3
enabled: true
description:
Approval rule for replacing sensitive SaaS systems with self-hosted alternatives
applies_to:
  category: open_source_replacement
conditions:
  contains_sensitive_data: true
  estimated_savings_annual_gte: 5000
decision:
  mode: require_approval
  approval_chain:
    - role: system_owner
    - role: security_reviewer
    - role: compliance_reviewer
  execution_mode: manual_after_approval
exceptions:
  - if:
    environment: sandbox
    then:
      mode: require_approval
      approval_chain:
        - role: system_owner
metadata:
  severity: high
```

28. Real UI Wireframes (Product Blueprint)

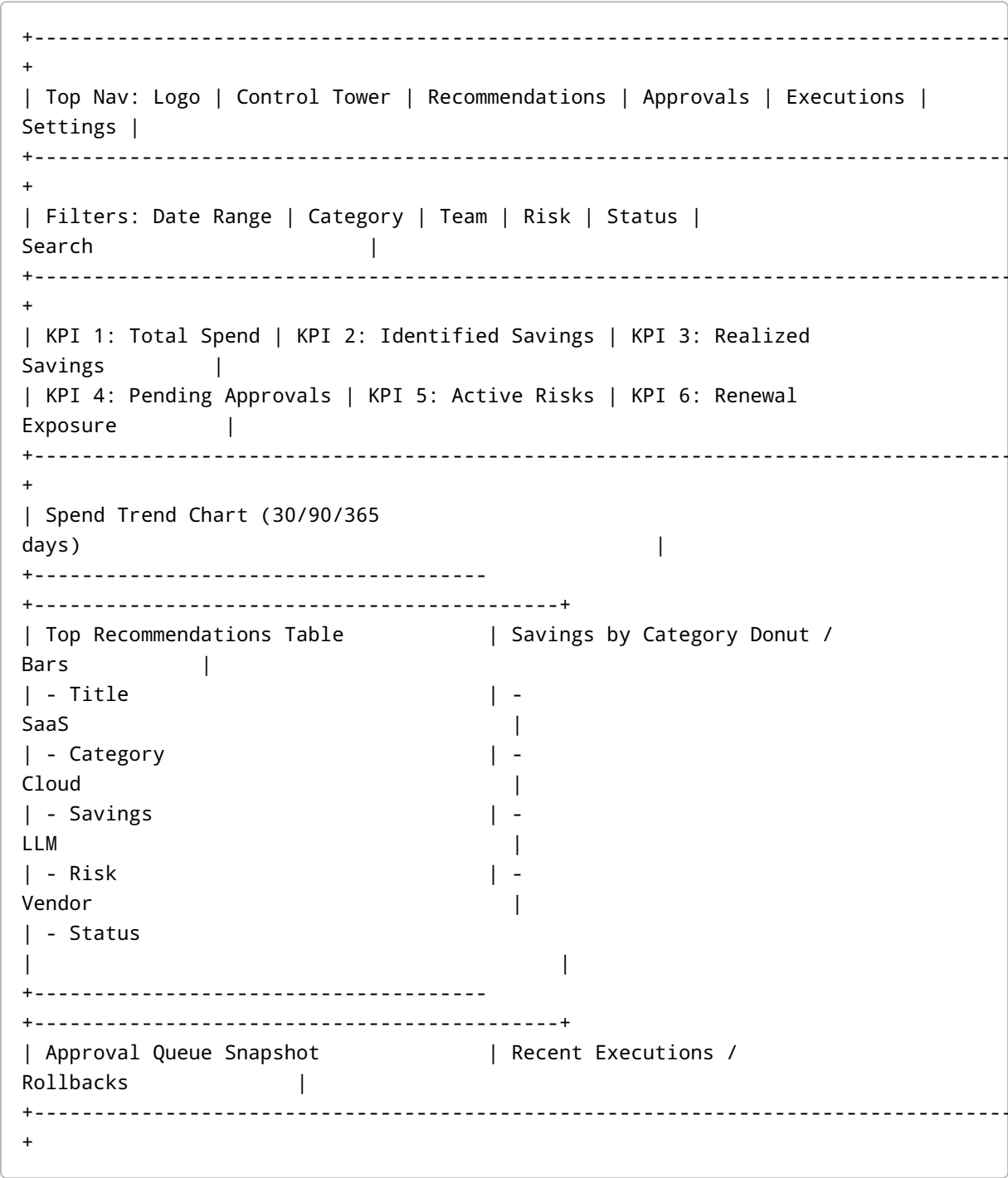
These are implementation-grade wireframe definitions for the first production UI.

28.1 Control Tower

Purpose

Single-pane view of spend, recommendations, approvals, and realized savings.

Layout



Key UI Components

- global filters
- metric cards
- trend visualization

- recommendation table
- approval queue widget
- execution activity feed

28.2 Recommendation Detail Page

Layout

+-----	
+	
Breadcrumb: Control Tower > Recommendations > Reduce inactive ExampleCRM seats	
+-----	
+	
Title: Reduce 180 inactive ExampleCRM seats	
Badges: SaaS Low Risk High Confidence Awaiting Approval	
+-----	
+	
Left Column (70%)	Right Column
(30%)	
+-----	
Summary	Financial
Impact	
Evidence	- Current
Cost	
Usage Signals	- Projected
Savings	
Impacted Teams / Owners	- Implementation
Cost	
Proposed Action Steps	-
Payback	
Skeptic Notes / Concerns	
Rollback Plan	Approval
Requirements	
Execution Preview	- Required
Roles	
	- SLA /
Deadline	
+-----	
+	
Tabs: Evidence Simulation Approvals Execution History Audit Log	

+-----+
+

28.3 Approval Inbox

Purpose

Dedicated workspace for human decision-makers.

Layout

+-----+
+
| Approval
Inbox |
+-----+
+
| Filters: My Approvals | Team | Due Today | Escalated | High
Risk |
+-----+
+
| Left Pane: Approval List | Right Pane: Selected
Approval |
+-----+
| [] Reduce ExampleCRM seats |
Title |
| [] Stop idle staging cluster | Why this
matters |
| [] Renew contract renegotiation |
Evidence |
| [] Replace analytics SaaS | Savings /
Risk |
| | Impacted systems /
users |
| | Rollback
plan |
| | Comments
thread |
| | [Approve] [Reject] [Defer]
[Escalate] |
+-----+
+

Design Rules

- approval actions must always show financial impact + risk together
- approver must see rollback plan before approving execution
- comment thread must be attached to decision history

28.4 Execution Center

Purpose

Monitor live and completed execution jobs.

Layout

+-----

+
| Execution
Center |

+-----

+
| Filters: Queued | Running | Completed | Failed | Rolled
Back |

+-----

+
| Execution Jobs
Table |
| Job ID | Recommendation | Mode | Status | Started | Completed |
Owner |

+-----

+
| Selected
Execution |
| - Execution plan
steps |
| - Current step
status |
| - Verification
checks |
| - Audit
trail |
| - [Pause] [Rollback] [View
Logs] |

+-----

+

28.5 Renewal Intelligence Workspace

Layout

```
+-----+
+
|
Renewals
|
+-----+
+
| Upcoming Contracts
Table
| Vendor | End Date | Auto Renew | Annual Commit | Owner |
Recommendation
+-----+
+
| Selected Vendor
Panel
| - Contract
summary
| - Usage vs
commit
| - Negotiation
leverage
| - Suggested target
price
| - Draft negotiation
memo
+-----+
+
```

28.6 Open Alternative Advisor

Layout

```
+-----+
+
| Open Alternative
Advisor
+-----+
+
```

Search / Filter: Category Compliance Need Hosting Preference Savings Range				
+-----				
+				
Current Tool		Open Option		TCO Delta Migration Risk
Action				
+-----				
+				
Analytics SaaS		Self-hosted Stack		-42% Medium
Review				
Internal Wiki SaaS		OSS Wiki		-58% Low
Review				
+-----				
+				
Detail				
Panel:				
- capability				
map				
- infra				
requirements				
- people / ops				
overhead				
- security				
notes				
- phased migration				
plan				
+-----				
+				

28.7 Navigation Model

Primary Navigation

- Control Tower
- Recommendations
- Approvals
- Executions
- Renewals
- Open Alternatives
- Reports
- Settings

Secondary Navigation

Contextual tabs within each module: - Summary - Evidence - Simulation - Approvals - Audit - History

29. Frontend Component Inventory

Core Components

- MetricCard
- SpendTrendChart
- RecommendationTable
- RecommendationDrawer
- ApprovalList
- ApprovalDecisionPanel
- ExecutionJobTable
- ExecutionTimeline
- RiskBadge
- SavingsImpactCard
- AuditLogTable
- ContractSummaryCard
- TCOCComparisonCard

Design System Notes

- enterprise-neutral visual style
- high information density with strong spacing
- explicit state badges
- immutable audit views
- no hidden approval context

30. Implementation Notes for Engineering

Backend Priorities

1. stable ontology + schema
2. recommendation pipeline
3. policy evaluation engine
4. approval routing
5. execution safety controls

Frontend Priorities

1. Control Tower
2. Recommendation details
3. Approval Inbox
4. Execution Center

5. Renewals workspace

AI / Agent Priorities

1. deterministic output schemas
 2. strong evidence grounding
 3. conservative risk classification
 4. challenge review before policy step
-

Final Note

With the SQL schema, agent prompts, tool contracts, policy DSL, and UI wireframes defined, this document is now a near-complete product + engineering blueprint for building **Autonomyx Optimize**.